САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ

ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №2

Проектирование и технический дизайн API

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Необходимо выполнить проектирование и технический дизайн API для бэкенд-приложения по выбранному варианту: приложение для бронирования столиков в ресторанах.

В рамках работы требуется описать функциональные и нефункциональные требования к приложению, выбрать формат реализации API, спроектировать эндпоинты в формате OpenAPI-схемы, описать тела запросов и ответов, а также продумать возможные ошибки, возвращаемые из API.

Результатом работы должна быть спецификация API, которую можно открыть в Swagger и использовать для обсуждения методов на защите.

Ход работы

1. Исходные данные

В качестве предметной области выбрано приложение для бронирования столиков в ресторанах. Пользователь может зарегистрироваться, войти в систему, найти ресторан по фильтрам, открыть страницу ресторана, посмотреть меню, фотографии, отзывы, забронировать столик и затем увидеть бронирование в личном кабинете.

Проектирование API выполнено на основе ранее разработанной ERD-схемы базы данных. В базе данных были выделены сущности пользователей, ресторанов, кухонь, фотографий, меню, столиков, бронирований и отзывов.

2. Функциональные требования

Функциональные требования определяют, какие действия пользователь и администратор должны иметь возможность выполнять через API.

Раздел	Описание требования
Авторизация	Регистрация пользователя, вход в систему, получение данных текущего пользователя.
Личный кабинет	Просмотр и редактирование профиля пользователя.
Поиск ресторанов	Получение списка ресторанов с фильтрацией по кухне, городу, району и ценовой категории.
Страница ресторана	Получение подробной информации о ресторане, его меню, фотографиях, столиках и отзывах.
Бронирования	Создание бронирования, просмотр истории бронирований, просмотр конкретного бронирования и отмена бронирования.
Отзывы	Просмотр отзывов о ресторане, создание, редактирование и удаление собственного отзыва.
Администрирование	Создание и изменение ресторанов, кухонь, меню, фотографий, столиков и статусов бронирований.

3. Нефункциональные требования

Нефункциональные требования описывают не отдельные действия пользователя, а общие свойства API: безопасность, стабильность, удобство использования и расширяемость.

- API должно использовать JSON как основной формат обмена данными.
- Для защищенных методов должна использоваться JWT-авторизация.
- Пароли пользователей должны храниться только в виде хеша, а не в открытом виде.
- Должна быть предусмотрена проверка ролей пользователя: user и admin.
- Входные данные должны валидироваться до выполнения бизнес-логики.
- Ошибки должны возвращаться в едином формате с понятным сообщением.
- Для списков ресторанов, отзывов и бронирований должна быть предусмотрена пагинация.
- Система не должна разрешать двойное бронирование одного столика на одно и то же время.
- Документация API должна быть представлена в формате OpenAPI и открываться через Swagger.

4. Выбор формата API

Для реализации был выбран формат REST API. Такой подход подходит для данного проекта, так как основные сущности базы данных удобно представить как ресурсы: пользователи, рестораны, кухни, меню, столики, бронирования и отзывы.

Backend For Frontend в данной работе не использовался, так как проект не предполагает несколько разных клиентов с принципиально различными API-контрактами. Для учебной реализации достаточно единого REST API, доступного через общие HTTP-методы GET, POST, PATCH и DELETE.

5. Группы эндпоинтов API

В результате проектирования API были выделены основные группы эндпоинтов:

Auth - Регистрация, вход и получение текущего пользователя.

```
POST /api/auth/register
POST /api/auth/login
GET /api/auth/me
```

Users - Личный кабинет пользователя.

```
GET /api/users/profile
PATCH /api/users/profile
```

Restaurants - Поиск, просмотр и управление ресторанами.

```
GET /api/restaurants
POST /api/restaurants
GET /api/restaurants/{restaurantId}
PATCH /api/restaurants/{restaurantId}
DELETE /api/restaurants/{restaurantId}
```

Cuisines - Работа с типами кухни.

```
GET /api/cuisines
POST /api/cuisines
PATCH /api/cuisines/{cuisineId}
DELETE /api/cuisines/{cuisineId}
```

Photos - Фотографии ресторанов.

```
GET /api/restaurants/{restaurantId}/photos
POST /api/restaurants/{restaurantId}/photos
DELETE /api/photos/{photoId}
```

Menu - Категории меню и блюда.

```
GET /api/restaurants/{restaurantId}/menu
POST /api/restaurants/{restaurantId}/menu/categories
POST /api/menu/categories/{categoryId}/items
PATCH /api/menu/items/{itemId}
DELETE /api/menu/items/{itemId}
```

Tables - Столики ресторанов.

```
GET /api/restaurants/{restaurantId}/tables
POST /api/restaurants/{restaurantId}/tables
PATCH /api/tables/{tableId}
DELETE /api/tables/{tableId}
```

Bookings - Создание, просмотр и изменение статуса бронирований.

```
POST /api/bookings
GET /api/bookings/my
GET /api/bookings/{bookingId}
PATCH /api/bookings/{bookingId}/cancel
PATCH /api/bookings/{bookingId}/status
```

Reviews - Отзывы о ресторанах.

```
GET /api/restaurants/{restaurantId}/reviews
POST /api/restaurants/{restaurantId}/reviews
PATCH /api/reviews/{reviewId}
DELETE /api/reviews/{reviewId}
```

Отдельная группа для таблицы restaurant_cuisines не выделялась, потому что это промежуточная таблица для связи многие-ко-многим между ресторанами и кухнями. Управление этой связью может выполняться через эндпоинты ресторанов или кухонь.

6. Примеры тел запросов и ответов

Для каждого метода в OpenAPI-схеме были описаны параметры, тела запросов, успешные ответы и ошибки. Ниже приведены примеры основных запросов.

Пример тела запроса на регистрацию пользователя:

```
{
  "name": "Ivan Petrov",
  "email": "user@example.com",
  "password": "User12345",
```

```
  "phone": "+79990000002"
}
```

Пример ответа при успешном входе пользователя:

```
{
  "token": "jwt_token",
  "user": {
    "id": 1,
    "name": "Ivan Petrov",
    "email": "user@example.com",
    "role": "user"
  }
}
```

Пример тела запроса на создание бронирования:

```
{
  "restaurant_id": 1,
  "table_id": 2,
  "guests_count": 4,
  "booking_start": "2026-06-12T18:00:00.000Z",
  "booking_end": "2026-06-12T20:00:00.000Z"
}
```

7. Возможные ошибки API

Для API были предусмотрены основные коды ошибок, которые позволяют клиентскому приложению корректно реагировать на неправильные данные, отсутствие авторизации или конфликт бизнес-логики.

Код ошибки	Причина
400 Bad Request	Переданы некорректные данные запроса, например отсутствует обязательное поле.
401 Unauthorized	Пользователь не авторизован или JWT-токен не передан.
403 Forbidden	Пользователь авторизован, но не имеет прав на выполнение действия.
404 Not Found	Запрашиваемый ресурс не найден: ресторан, бронирование, отзыв, столик и т.д.
409 Conflict	Конфликт данных, например email уже зарегистрирован или столик уже забронирован на выбранное время.

Единый формат ошибки можно представить следующим образом:

```
{
  "statusCode": 409,
  "error": "Conflict",
  "message": "Столик уже забронирован на выбранное время"
}
```

Вывод

В ходе работы был выполнен технический дизайн REST API для приложения бронирования столиков в ресторанах. Были определены функциональные и нефункциональные требования, выбран формат REST API, выделены группы эндпоинтов и описаны основные запросы и ответы.

Также были продуманы возможные ошибки API, включая ошибки валидации, авторизации, доступа, отсутствия ресурса и конфликта бронирования. Полученная OpenAPI-спецификация может использоваться как основа для реализации серверной части приложения и демонстрации API через Swagger